

FinQuest AI Features: Architecture & Technology Roadmap

A Detailed Technical Document

Table of Contents

1. Introduction
 2. AI in FinQuest: Current State
 3. Vision for AI in FinQuest
 4. AI Data Pipeline
 5. Planned AI Features
 6. Technology Choices
 7. Architecture for AI Insights
 8. Spending Pattern Analysis
 9. Budget Recommendation Engine
 10. Anomaly Detection
 11. Personalized Saving Tips
 12. Implementation Roadmap
 13. Privacy & Security Considerations
 14. Evaluation Metrics
 15. Integration with Backend
 16. Frontend AI Experience
 17. Future Possibilities
 18. Conclusion
-

1. Introduction

Artificial Intelligence in FinQuest is designed to transform raw financial data into actionable insights. While the current version of FinQuest focuses on tracking, budgeting, and gamification, the architecture has been designed with AI expansion in mind.

This document explains:

- What AI features are planned
 - How they will be architected
 - Which technologies will power them
 - Why specific libraries and approaches were chosen
 - How AI integrates with the existing backend and frontend
-

2. AI in FinQuest: Current State

2.1 Existing AI Infrastructure

The backend already includes placeholder models for AI features:

```
backend/app/models/ai.py
├── AIInsight
│   ├── user_id
│   ├── type
│   ├── confidence
│   ├── data
│   └── created_at
└── SpendingPattern
    ├── user_id
    ├── category_id
    ├── avg_monthly_spend
    ├── trend_slope
    ├── seasonality_score
    └── created_at
```

These models are currently not heavily used but exist to support future AI capabilities.

2.2 Why Placeholder Models Were Added Early

Adding the AI models early provides:

- **Schema Stability:** Database migrations won't require major changes later
- **Clear Extension Points:** Developers know where AI data belongs
- **Future-Proofing:** The system is ready for AI features as soon as algorithms are implemented

3. Vision for AI in FinQuest

The goal of AI in FinQuest is to make users better at managing money without requiring them to be financial experts. Specifically, AI will:

Goal	How AI Helps
Understand Spending	Identify patterns, categories, and trends
Predict Future Behavior	Forecast monthly expenses and savings
Detect Problems Early	Flag unusual transactions or budget overruns
Recommend Actions	Suggest budgets, saving targets, and category adjustments
Personalize Experience	Adapt insights to each user's financial behavior

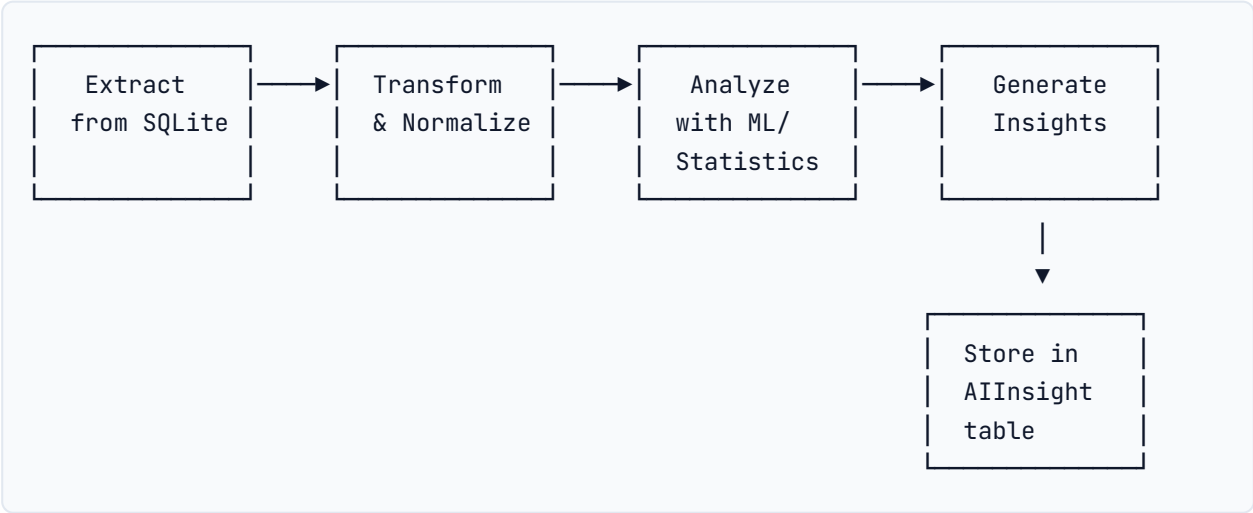
4. AI Data Pipeline

4.1 Data Sources

The AI engine will consume data from the existing database:

Source	Data
Transactions	Amounts, categories, dates, descriptions
Budgets	Budget amounts, periods, alert thresholds
Goals	Target amounts, deadlines, contributions
User Profile	Income level, spending history, preferences
Gamification	Streaks, achievements, XP patterns

4.2 Pipeline Stages



4.3 Why This Pipeline?

- **Separation of Concerns:** AI processing doesn't block user actions
- **Caching:** Insights are stored and reused until data changes
- **Scalability:** Pipeline can run on a schedule or in background workers

5. Planned AI Features

5.1 Feature Roadmap

Feature	Priority	Description
Spending Pattern Analysis	High	Identify monthly spending trends by category
Budget Recommendations	High	Suggest budget amounts based on historical spending
Anomaly Detection	Medium	Flag unusual transactions or spending spikes
Savings Forecasting	Medium	Predict how long to reach a savings goal
Smart Categorization	Medium	Auto-suggest categories for new transactions
Personalized Tips	Low	Generate money-saving tips based on behavior

6. Technology Choices

6.1 Core AI Libraries

Library	Purpose	Why Chosen
scikit-learn	Machine learning algorithms	Simple, well-documented, integrates with NumPy
NumPy	Numerical computation	Foundation for data processing and linear algebra
SciPy	Statistical analysis	Advanced statistics, hypothesis testing, optimization
pandas	Data manipulation	Easy transformation of transaction data into DataFrames
statsmodels	Time series analysis	Trend decomposition, seasonality detection

6.2 Why Not Deep Learning?

For FinQuest's current scope, traditional machine learning is sufficient because:

- **Less Data Required:** scikit-learn works well with thousands of transactions
- **Explainable:** Users should understand why a budget was recommended
- **Faster Training:** Models train in seconds, not hours
- **Lower Resource Usage:** Runs on a small VPS without a GPU

Deep learning (TensorFlow/PyTorch) would be considered for:

- Natural language processing of transaction descriptions
- Advanced anomaly detection on millions of transactions
- Image recognition for receipt scanning

6.3 Why scikit-learn Specifically?

scikit-learn is the Swiss Army knife of Python machine learning:

Reason 1: Consistent API

Every model follows the same pattern:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Reason 2: Comprehensive Toolkit

scikit-learn includes:

- Regression and classification
- Clustering (K-Means, DBSCAN)
- Dimensionality reduction (PCA)
- Model evaluation metrics
- Preprocessing utilities

Reason 3: Production Ready

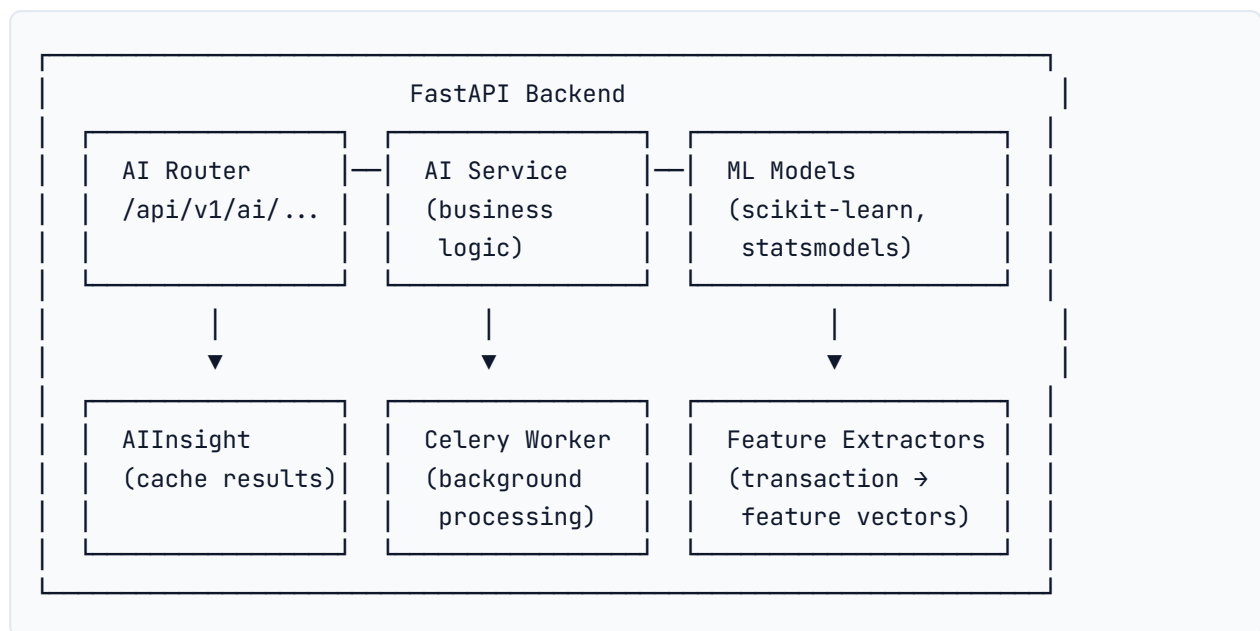
Models can be serialized with `joblib` and loaded quickly at runtime.

Reason 4: Excellent Documentation

The scikit-learn documentation is among the best in the Python ecosystem, making it ideal for a student project.

7. Architecture for AI Insights

7.1 Proposed Architecture



7.2 Why a Separate AI Service?

- **Modularity:** AI logic is isolated from core financial operations
- **Testability:** Models can be tested independently
- **Scalability:** AI processing can be moved to background workers

- **Maintainability:** Data scientists can work on models without touching routers

8. Spending Pattern Analysis

8.1 Problem

Users want to understand where their money goes over time. Simple totals don't reveal trends.

8.2 Approach

1. **Aggregate** transactions by month and category
2. **Calculate** moving averages for smoothing
3. **Detect** trends using linear regression
4. **Identify** seasonality (e.g., higher spending on weekends or holidays)
5. **Store** results in `SpendingPattern` model

8.3 Algorithm Example

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Monthly spending for a category
months = np.array([[1], [2], [3], [4], [5], [6]])
amounts = np.array([200, 220, 210, 250, 240, 260])

model = LinearRegression()
model.fit(months, amounts)

trend_slope = model.coef_[0] # Positive = increasing spending
avg_spend = np.mean(amounts)
```

8.4 Insight Generation

Trend	Insight
Increasing spending	"Your Food spending has increased 15% over 6 months"
Seasonal spike	"You typically spend 30% more on Entertainment in December"
Stable spending	"Your Transport spending is consistent month-to-month"

9. Budget Recommendation Engine

9.1 Problem

New users don't know how much to budget for each category.

9.2 Approach

1. Analyze 3-6 months of historical spending per category
2. Calculate average monthly spend and standard deviation
3. Recommend budget = average + small buffer
4. Cap recommendations at a percentage of income

9.3 Formula

```
recommended_budget = avg_monthly_spend + (1.5 * std_deviation)

# Ensure it's not more than 30% of monthly income
max_budget = monthly_income * 0.30
recommended_budget = min(recommended_budget, max_budget)
```

9.4 Why This Approach?

- **Data-Driven:** Based on actual user behavior, not generic rules
- **Conservative:** Slightly above average to avoid frequent overspending
- **Income-Aware:** Prevents budgets from consuming too much income

10. Anomaly Detection

10.1 Problem

Users should be alerted to unusual transactions that might be fraud, mistakes, or overspending.

10.2 Approach

Use statistical anomaly detection:


```

from scipy import stats
import numpy as np

# Historical amounts for a category
history = np.array([45, 50, 48, 52, 47, 200]) # 200 is unusual

z_scores = np.abs(stats.zscore(history))
threshold = 2.5

anomalies = history[z_scores > threshold]

```

10.3 Why Statistical Methods?

- **No Training Required:** Works immediately with user's own data
- **Explainable:** "This transaction is 3 standard deviations above your average"
- **Lightweight:** Computes in milliseconds

11. Personalized Saving Tips

11.1 Problem

Generic advice like "spend less" is not helpful. Users need personalized, actionable tips.

11.2 Approach

Rule-based system enhanced with spending patterns:

Condition	Generated Tip
High food spending	"Cooking at home 2 more days per week could save you \$80/month"
Unused subscriptions	"You spent \$50 on Entertainment last month but only logged in twice"
Goal behind schedule	"Increase monthly savings by \$25 to reach your Emergency Fund goal on time"
Frequent small purchases	"Your \$5 daily coffee habit adds up to \$150/month"

11.3 Why Rule-Based + ML?

- **Rule-Based:** Guarantees relevant, safe advice
- **ML-Enhanced:** Personalizes amounts and priorities based on behavior

12. Implementation Roadmap

12.1 Phase 1: Statistical Insights (Current → 2 weeks)

- Implement spending pattern analysis
- Store results in `SpendingPattern` model
- Display basic insights on dashboard

12.2 Phase 2: Budget Recommendations (2–4 weeks)

- Build recommendation engine
- Add "Suggested Budget" button to budget creation form
- A/B test recommendation acceptance

12.3 Phase 3: Anomaly Detection (4–6 weeks)

- Implement z-score based anomaly detection
- Add notification alerts for unusual transactions
- Allow users to mark false positives

12.4 Phase 4: Predictive Features (6+ weeks)

- Savings goal forecasting
 - Expense prediction for next month
 - Smart categorization using NLP
-

13. Privacy & Security Considerations

13.1 Data Privacy

- All AI processing uses **on-device or server-side computation** — no third-party AI services
- User financial data never leaves the FinQuest backend
- Insights are personalized but anonymized in logs

13.2 Security

- AI endpoints require authentication
- Rate limiting prevents abuse of expensive computations

- Input validation prevents injection into ML pipelines

14. Evaluation Metrics

14.1 How to Measure AI Success

Metric	Description	Target
Recommendation Acceptance Rate	% of users who adopt suggested budgets	> 40%
Anomaly Precision	% of flagged transactions that are actually unusual	> 80%
Insight Engagement	% of users who view AI insights weekly	> 60%
Prediction Accuracy	MAE of monthly expense forecasts	< 15% of average spend
User Retention	Users returning after receiving insights	+20% vs control

15. Integration with Backend

15.1 New AI Router

```
# Proposed: backend/app/routers/ai.py
from fastapi import APIRouter

router = APIRouter(prefix="/api/v1/ai", tags=["ai"])

@router.get("/insights")
def get_insights(current_user = Depends(get_current_active_user)):
    # Generate or fetch cached insights
    return insights_service.get_user_insights(current_user.id)

@router.get("/budget-recommendation")
def recommend_budget(category_id: int, current_user = Depends(...)):
    return budget_recommender.recommend(current_user.id, category_id)
```

15.2 Why This Integration?

- Follows existing backend patterns
- Uses the same authentication and response wrapping
- Easy to extend with new AI endpoints

16. Frontend AI Experience

16.1 AI Dashboard Section

A new "AI Insights" card on the dashboard showing:

- Top spending trend
- Unusual transactions this month
- Personalized tip of the week

16.2 Budget Creation Assistant

When creating a budget, show a "Suggest Budget" button that calls the recommendation API.

16.3 Notification Integration

AI-generated insights appear as in-app notifications using the existing notification system.

17. Future Possibilities

17.1 Advanced AI Directions

Direction	Technology	Use Case
Natural Language Processing	spaCy / transformers	Auto-categorize transaction descriptions
Receipt Scanning	OCR (Tesseract)	Extract transaction data from photos
Reinforcement Learning	Custom RL agent	Optimize budget allocation over time
Fraud Detection	Isolation Forest	Detect suspicious transaction patterns
Voice Assistant	Whisper + LLM	"How much did I spend on food this month?"

17.2 Large Language Models (LLMs)

LLMs like GPT could power:

- Natural language queries about spending
- Conversational financial advice
- Automatic report generation

However, LLMs would only be used:

- With explicit user consent
 - For non-sensitive aggregations
 - With local or trusted hosted models to preserve privacy
-

18. Conclusion

AI is the next major evolution for FinQuest. The current backend architecture already provides the data foundation, and the chosen technologies (scikit-learn, NumPy, SciPy, pandas) offer a practical, explainable path to intelligent features.

By starting with statistical insights and rule-based recommendations, FinQuest can deliver immediate value without requiring massive datasets or expensive infrastructure. As the user base grows, the architecture supports more advanced machine learning and even large language models.

The guiding principle is simple: **AI should make financial management easier, not more complicated.**

Document generated: May 2026

FinQuest v1.0.0 — AI Features Architecture & Technology Roadmap